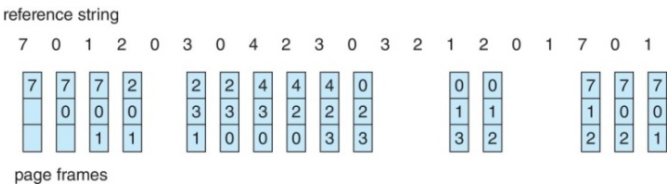
الگوریتم جایگزینی FIFO برای هر یک از صفحات، زمان آوردن آنها به حافظه را نسبت می‌دهد. زمانی که جایگزینی صفحه لازم می‌شود، قدیمی‌ترین صفحه انتخاب می‌شود. توجه داشته باشید که ضرورتی ندارد زمان آوردن صفحه به حافظه را ثبت کنیم. برای این کار می‌توانیم یک صف FIFO ایجاد کنیم تا تمامی صفحات داخل حافظه را نگه بدارد. صفحه‌ای را جایگزین می‌کنیم که سر صف باشد. زمانی هم که صفحه‌ای را به حافظه می‌آوریم، آن را در ته صف قرار می‌دهیم.

برای رشته‌ی مراجعه‌ی نمونه‌ی خودمان، هر سه قاب در ابتدا خالی هستند. سه مراجعه‌ی نخست، (۷، ۰، ۱) باعث خطای صفحه شده و صفحات مزبور را به داخل این سه قاب می‌آورند. مراجعه‌ی بعدی (۲) جایگزین صفحه‌ی ۷ می‌شود، چون صفحه‌ی ۷ قبل از همه آورده شده است. از آنجایی که مراجعه‌ی بعدی ۰ است و ۰ از قبل در حافظه حضور دارد، این مراجعه خطای صفحه نخواهد داشت. اولین مراجعه‌ای که به صفحه‌ی ۳ صورت می‌گیرد صفحه‌ی ۰ را برای جایگزینی انتخاب می‌کند، زیرا اکنون در اول صف قرار دارد. به خاطر این جایگزینی است که مراجعه‌ی بعدی به ۰، خطای صفحه خواهد داد. و در ادامه، صفحه‌ی ۰ به جای صفحه‌ی ۱ جایگزین می‌شود. این روال همچنان که در شکل ۱۰-۱۲ نشان داده می‌شود تداوم می‌یابد. در هر بار که خطای صفحه رخ می‌دهد، صفحات موجود هر سه قاب را نشان می‌دهیم. تعداد خطاها درمجموع ۱۵ مورد خواهد بود.

فهم و برنامه‌نویسی الگوریتم جایگزینی صفحه‌ی FIFO آسان است. با این وجود، کارایی آن همواره خوب نیست. از یک سو ممکن است صفحه‌ای را جایگزین کند که یک ماژول مقداردهی اولیه بوده است که خیلی وقت پیش به کار رفته و دیگر لازم نیست. از طرفی، امکان دارد حاوی متغیری باشد که در ابتدا مقداردهی می‌شود و مدام استفاده دارد.

توجه داشته باشید که حتی اگر صفحه‌ای را برای جایگزینی انتخاب نماییم که استفاده‌ی اکیدی دارد، باز هم همه چیز درست کار می‌کند. بعد از جایگزینی صفحه‌ی جدید با صفحه‌ای که مؤکد استفاده دارد، تقریباً بلافاصله خطای صفحه‌ای رخ می‌دهد و همان صفحه را بازیابی می‌کند. در اینجا، بایستی صفحه‌ی دیگری جایگزین شود تا صفحه‌ای که مؤکد استفاده می‌شد به حافظه آورد شود. بنابراین، با انتخاب جایگزین بد، میزان خطای صفحه افزایش یافته و اجرای پردازه کند می‌شود. با این وجود، این امر موجب اجرای نادرست نمی‌شود.



شکل ۱۰-۱۲ الگوریتم جایگزینی صفحه‌ی FIFO.

برای روشن‌شدن مسائلی که در رابطه با جایگزینی FIFO وجود دارد، رشته‌ی مراجعه‌ی زیر را در نظر می‌گیریم:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

شکل ۱۰-۱۳، منحنی خطاهای صفحه‌ی این رشته‌ی مراجعه را در مقایسه با تعداد قاب‌های موجود نشان می‌دهد. توجه کنید که تعداد خطاهای صفحه برای چهار قاب (ده مورد) از تعداد خطاهای صفحه با سه قاب بیشتر است (نه مورد)! این نتیجه‌ی غیر منتظره به **ناهنجاری Belady** معروف است: در برخی الگوریتم‌های جایگزینی صفحه، نرخ خطای صفحه می‌تواند با *افزایش* تعداد قاب‌های تخصیصی، افزایش پیدا کند. انتظار داریم که اگر حافظه‌ی بیشتری در اختیار پردازه قرار بگیرد کارایی آن بهبود پیدا کند. محققان در یک تحقیق اخیر متوجه شدند که این فرض نمی‌تواند همواره درست باشد. پیامد این کار، کشف ناهنجاری Belady بود.

۱۰-۴-۳ جایگزینی صفحه‌ی بهینه

یکی از نتایج کشف ناهنجاری Belady این بود که دنبال یک **الگوریتم جایگزینی صفحه‌ی بهینه**[1] باشیم–الگوریتمی که در میان تمامی الگوریتم‌ها، پایین‌ترین نرخ خطای صفحه را داشته و هرگز گرفتار ناهنجاری Belady نشود. چنین الگوریتمی وجود دارد و الگوریتم OPT یا MIN نامیده می‌شود. این الگوریتم به سادگی به صورت زیر بیان می‌شود:

صفحه‌ای را جایگزین کن که طولانی‌ترین دوره‌ی زمانی استفاده نخواهد داشت.

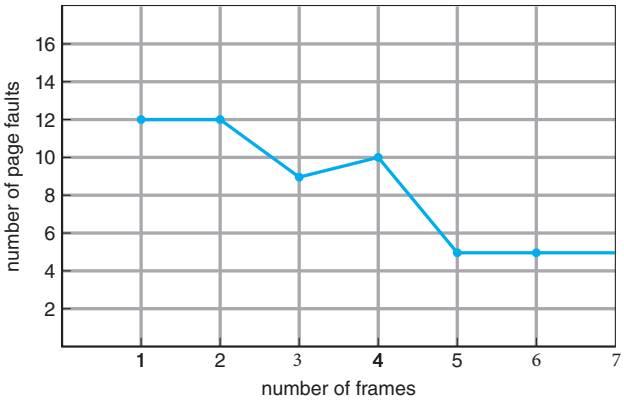
استفاده از این الگوریتم جایگزینی صفحه تضمین می‌نماید که میزان خطای صفحه با داشتن تعداد ثابتی از قاب‌ها، پایین‌ترین مقدار ممکن را خواهد داشت.

برای مثال، اعمال الگوریتم جایگزینی صفحه‌ی بهینه بر روی رشته‌ی مراجعه‌ی نمونه‌ی خودمان، فقط به تعداد نه مورد خطای صفحه خواهد داد که در شکل ۱۰-۱۴ نشان داده می‌شود. سه مراجعه‌ی اول خطای صفحه می‌دهند که سه قاب خالی را پر می‌کنند. مراجعه‌ی صفحه‌ی ۲، صفحه‌ی ۷ را جایگزین انتخاب می‌کند، چون صفحه‌ی ۷ تا مراجعه‌ی هیجدهم استفاده نخواهد داشت، در حالی که، صفحه‌ی ۰ در مراجعه‌ی پنجم و صفحه‌ی ۱ در مراجعه‌ی چهاردهم مورد استفاده قرار می‌گیرند. مراجعه به صفحه‌ی ۳، صفحه‌ی ۱ را جایگزین انتخاب می‌کند، چون صفحه‌ی ۱ در بین صفحات دیگر درون حافظه، آخرین صفحه‌ای است که دوباره مورد استفاده قرار خواهد گرفت. جایگزینی بهینه فقط با نه خطای صفحه، خیلی بهتر از الگوریتم FIFO می‌باشد که تعداد خطای آن پانزده مورد است. (اگر از سه خطای اول چشم‌پوشی کنیم که در هر الگوریتم اتفاق می‌افتند، در این صورت جایگزینی بهینه از جایگزینی FIFO دومرتبه بهتر عمل می‌کند.) در حقیقت، هیچ یک از الگوریتم‌های جایگزینی نمی‌توانند این رشته‌ی مراجعه را با داشتن سه قاب با کمتر از نه خطای صفحه پردازش نمایند.

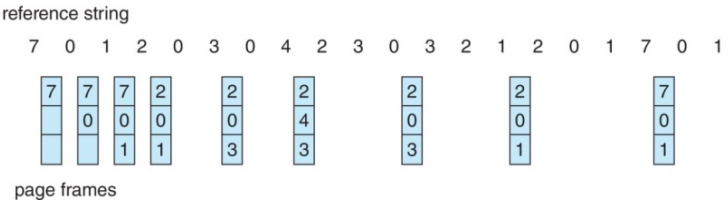
متأسفانه، پیاده‌سازی الگوریتم جایگزینی بهینه مشکل است، چون به دانش آتی از رشته‌ی مراجعه نیاز دارد. (به مشکل مشابهی هم در بخش ۵-۳-۲ در رابطه با الگوریتم زمانبندی SJF پردازنده برخورد کردیم.) در نتیجه، الگوریتم بهینه اساساً برای قیاس مطالعاتی استفاده می‌شود.

1. Optimal page-replacement algorithm

برای نمونه، ممکن است مفید باشد که بدانیم هرچند یک الگوریتم جدید بهینه نیست، اما در بدترین حالت، ۱۲/۳ درصد و به طور متوسط، ۴/۷ درصد بهینه می‌باشد.



شکل ۱۰-۱۳ منحنی خطای صفحه برای جایگزینی FIFO روی یک رشته‌ی مراجعه.



شکل ۱۰-۱۴ الگوریتم جایگزینی صفحه‌ی بهینه.

۱۰-۴-۴ جایگزینی صفحه‌ی LRU

اگر پیاده‌سازی الگوریتم بهینه امکان‌پذیر نیست، شاید تقریبی از الگوریتم بهینه ممکن باشد. تفاوت کلیدی میان الگوریتم‌های FIFO و OPT (به غیر از این که، نگاه آنها از نظر زمان به پس و پیش است) در این است که الگوریتم FIFO زمانی را به کار می‌برد که صفحه به داخل حافظه آورده می‌شود، حال آنکه الگوریتم OPT زمانی را به کار می‌گیرد که صفحه مورد استفاده قرار خواهد گرفت. اگر از گذشته‌ی اخیر به عنوان تقریبی از آینده‌ی نزدیک بهره بگیریم، در این صورت می‌توانیم صفحه‌ای را جایگزین کنیم که برای دوره‌ی طولانی‌تری مورد استفاده قرار نگرفته است. این رویکرد، **الگوریتم کمتر اخیراً به کار رفته**[1] (**LRU**) می‌باشد.

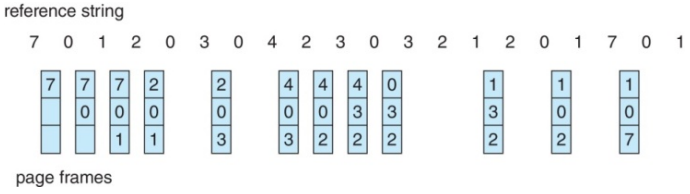
جایگزینی LRU برای هر صفحه، زمانی را که صفحه آخرین استفاده را داشته نسبت می‌دهد. موقعی که قرار است جایگزینی صفحه انجام شود، LRU صفحه‌ای را انتخاب می‌کند که برای دوره‌ی طولانی‌تری استفاده نداشته است. تصوری که از این راهبرد می‌توان داشت، این است که همانند الگوریتم جایگزینی صفحه‌ی بهینه عمل می‌کند، با این تفاوت که از نظر زمانی به جای نگاه

1. Least-Recently-Used algorithm

به پیش به عقب می‌نگرد (عجیب‌تر این که، اگر S^R معکوس رشته‌ی مراجعه‌ی S باشد، در این صورت نرخ خطای صفحه با الگوریتم OPT بر روی S با نرخ خطای صفحه‌ی الگوریتم OPT بر روی S^R یکسان خواهد بود. به طور مشابه، نرخ خطای صفحه‌ی الگوریتم LRU بر روی S با نرخ خطای صفحه‌ی الگوریتم LRU بر روی S^R یکی خواهد بود.)

حاصل اعمال جایگزینی LRU بر روی رشته‌ی مراجعه‌ی نمونه‌ی خودمان در شکل ۱۰–۱۵ نشان داده می‌شود. تعداد خطای صفحه که در الگوریتم LRU تولید می‌شود ۱۲ مورد است. توجه کنید که پنج خطای نخست همانند خطاهای جایگزینی بهینه می‌باشند. با این وجود، لحظه‌ای که صفحه‌ی ۴ رجوع می‌گردد، جایگزینی LRU از میان سه قاب موجود درون حافظه متوجه می‌شود که صفحه‌ی ۲، اخیراً کمتر استفاده داشته است. بنابراین، الگوریتم LRU بی‌آنکه دانشی از چگونگی استفاده‌ی صفحه‌ی ۲ داشته باشد آن را برای جایگزینی انتخاب می‌کند. در ادامه، زمانی که برای صفحه‌ی ۲ خطا گرفته می‌شود، الگوریتم LRU آن را جایگزین صفحه‌ی ۳ می‌سازد، چرا که فعلاً در میان قاب‌های موجود حافظه، ۳ صفحه‌ای است که اخیراً کمترین استفاده را داشته است. به رغم این مسائل، الگوریتم LRU با تعداد ۱۲ خطای صفحه، باز هم خیلی بهتر از جایگزینی FIFO با ۱۵ خطا می‌باشد.

سیاست LRU غالباً به عنوان یک الگوریتم جایگزینی صفحه استفاده می‌شود و به نظر هم خوب است. مسأله‌ی اصلی، چگونگی پیاده‌سازی جایگزینی LRU می‌باشد. الگوریتم جایگزینی صفحه‌ی LRU می‌تواند نیاز قابل توجهی به یاری سخت‌افزار داشته باشد. مسأله این است که ترتیبی برای قاب‌ها با تعریف از روی آخرین زمان استفاده تعیین کنیم. دو نوع پیاده‌سازی ممکن عبارتند از:

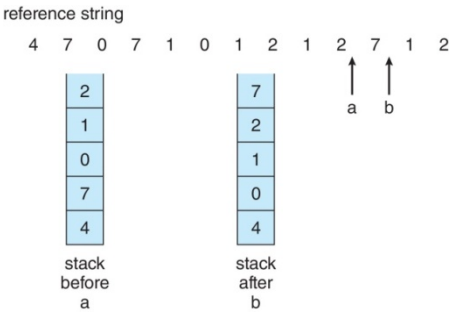


شکل ۱۰–۱۵ الگوریتم جایگزینی صفحه‌ی LRU.

- **شمارنده‌ها:** در ساده‌ترین حالت، به هر یک از واردهای جدول صفحه یک فیلد زمان استفاده نسبت می‌دهیم و یک ساعت منطقی یا شمارنده به پردازنده اضافه می‌کنیم. ساعت با هر بار مراجعه به حافظه یک واحد افزایش می‌یابد. هر بار که یک مراجعه به صفحه‌ای صورت می‌گیرد محتوای ثبات ساعت در فیلد زمان استفاده‌ی آن صفحه واقع در واردهی جدول صفحه کپی می‌شود. به این ترتیب، همیشه «زمان» آخرین مراجعه به هر یک از صفحات را خواهیم داشت. صفحه‌ای را برای جایگزینی انتخاب می‌کنیم که کوچکترین مقدار زمان را دارد. این طرح برای هر بار دسترسی حافظه به جستجوی جدول صفحه برای یافتن صفحه‌ی LRU و یک بار نوشتن حافظه (فیلد زمان استفاده در جدول صفحه) نیاز دارد. این زمان‌ها بایستی در

صورت تعویض جدول صفحات (بـه خـاطر زمانبنـدی پردازنـده) نیـز حفـظ شـوند. سرریزی ساعت نیز بایستی مد نظر قرار بگیرد.

- **پشته**: روش دیگر پیاده‌سازی جایگزینی LRU، نگه‌داری پشته‌ای از شماره صـفحات است. هر بار که یک صفحه مراجعه می‌شود، از پشته حذف شده و در بالای آن قرار می‌گیرد. به این ترتیب، صفحه‌ای که بیشترین استفاده را اخیراً داشته اسـت همـواره در بالای پشته قرار می‌گیرد و صفحه‌ای که کمترین استفاده را اخیراً داشته است در ته پشته واقع می‌شود (شکل ۱۰-۱۶). چون لازم اسـت کـه واردهـا از میـان پشـته حذف شوند بهترین کار برای پیاده‌سازی ایـن رویکـرد آن اسـت کـه از یـک لیسـت پیوندی دوطرفه با اشاره‌گری به سر و یک اشاره‌گر به ته آن استفاده شـود. در ایـن صورت، در بدترین حالت، حذف یک صفحه و قراردادن آن در بـالای پشـته، تغییـر شش اشاره‌گر را لازم خواهد داشت. هرچند که هزینه‌ی به روز رسانی پشته انـدکی بیشتر است، اما برای جایگزینی، نیازی به جستجو نخواهد داشت؛ اشاره‌گر ته پشـته که به قعر آن اشاره دارد، صفحه‌ی LRU را مشخص می‌کند. ایـن روش مخصوصـاً برای پیاده‌سازی نرم‌افزاری یا ریزکد جایگزینی LRU مناسب است.



شکل ۱۰-۱۶ استفاده از یک پشته برای ثبت اخیرترین مراجعات صفحه.

جایگزینی LRU، مثل جایگزینی بهینه، ناهنجاری Belady ندارد. هر دو به رده‌ای از الگوریتم‌های جایگزینی صفحه، به نام **الگوریتم‌های پشته** تعلق دارند و هرگز ناهنجاری Belady به آنها راه ندارد. الگوریتم پشته، الگوریتمی است که بتوان نشان داد مجموعه‌ی صفحات درون حافظه‌اش با n قاب، همواره زیرمجموعه‌ی مجموعه صفحاتی خواهند بود که با $n + 1$ قاب در داخل حافظه قرار می‌گیرند. این مجموعه صفحات درون حافظه در مورد جایگزینی LRU، اخیرترین n صفحه‌ای خواهند بود که مراجعه داشته‌اند. اگر تعداد قاب‌ها افزایش یابد، این n صفحه همچنان اخیرترین صفحات مراجعه شده خواهند بود و لذا همچنان در حافظه می‌مانند.

توجه داشته باشید که هیچ یک از پیاده‌سازی‌های LRU بدون کمک سخت‌افزار، جدا از ثبات‌های معمول TLB، مقدور نخواهند بود. در هر مراجعه به حافظه، بایستی فیلدهای ساعت یا پشته به روز شوند. اگر به ناچار از یک وقفه برای هر مراجعه استفاده کنیم تا اجازه بدهیم که به روز رسانی این ساختمان‌داده‌ها توسط نرم‌افزار انجام شود، مراجعات حافظه حداقل با ضریبی از

ده کند خواهند شد، از این رو هر یک از پردازه‌ها نیز با ضریبی از ده کندتر می‌شوند. سیستم‌های معدودی را می‌توان یافت که در مدیریت حافظه بتوانند سرباری تا این سطح را تحمل کنند.

۱۰-۴-۵ جایگزینی صفحه‌ی تقریب LRU

سیستم‌های کامپیوتری زیادی نیستند که سخت‌افزار کافی پشتیبانی جایگزینی صفحه‌ی واقعی LRU را داشته باشند. در حقیقت، برخی سیستم‌ها هیچ‌گونه پشتیبانی سخت‌افزاری ندارند و بایستی از سایر الگوریتم‌های جایگزینی صفحه (نظیر الگوریتم FIFO) استفاده شود. با این همه، سیستم‌های بسیاری هستند که اندک کمکی به شکل **بیت مراجعه** فراهم می‌نمایند. هر بار که صفحه‌ای مراجعه می‌گردد (با خواندن و یا نوشتن هر یک از بایت‌های آن صفحه) بیت مراجعه‌ی آن صفحه توسط سخت‌افزار یک می‌شود. بیت‌های مراجعه به هر یک از واردهای جدول صفحه منتسب می‌شوند.

در ابتدا، تمامی بیت‌ها توسط سیستم‌عامل پاک می‌شوند (به ۰). همین که پردازه‌ای اجرا می‌شود، بیت مربوط به هر یک از صفحاتی که مراجعه دارند توسط سخت‌افزار برپا می‌شوند (به ۱). بعد از مدت زمانی می‌توانیم با بررسی بیت‌های مراجعه، صفحاتی را که مورد استفاده قرار گرفته‌اند تعیین کنیم، هرچند که ترتیب استفاده از آنها را نمی‌دانیم. این اطلاعات پایه‌گذار بسیاری از الگوریتم‌های جایگزینی صفحه‌ای است که جایگزینی LRU را تقریب می‌زنند.

۱۰-۴-۵-۱ الگوریتم بیت‌های مراجعه‌ی اضافی

با ثبت بیت‌های مراجعه در فواصل زمانی منظم می‌توانیم اطلاعات بیشتری در رابطه با ترتیب مراجعات صفحات بدست آوریم. یک بایت ۸ بیتی برای هر یک از صفحات جدول در داخل حافظه نگه می‌داریم. در فواصل زمانی منظمی، وقفه‌ای از تایمر (برای مثال، در هر ۱۰۰ میلی‌ثانیه)، کنترل را به سیستم‌عامل منتقل می‌کند. سیستم‌عامل، بیت مراجعه‌ی هر صفحه را به بیت با ارزش بایت ۸ بیتی انتقال می‌دهد که با این کار، سایر بیت‌ها هم ۱ بیت به سمت راست شیفت پیدا کرده و از کم ارزش‌ترین بیت نیز چشم‌پوشی می‌شود. این ثبات‌های ۸ بیتی، سابقه‌ی استفاده‌ی هر یک از صفحات را در هشت دوره‌ی زمانی اخیر مشخص می‌کنند. برای مثال، اگر مقدار ثبات انتقالی برابر 00000000 باشد، در این صورت، صفحه در هشت دوره‌ی زمانی اخیر مورد استفاده قرار نگرفته است؛ صفحه‌ای که در هر دوره حداقل یکبار مراجعه می‌شود، ثبات انتقالی آن مقدار 11111111 را خواهد داشت. یک صفحه با مقدار ثبات سابقه‌ی 11000100 در مقایسه با مقدار ثبات 01110111، اخیراً استفاده‌ی بیشتری داشته است. اگر این بایت‌های ۸ بیتی را اعداد صحیح بدون علامت تعبیر کنیم، صفحه‌ای که مقدار عددی آن کوچکترین است صفحه‌ی LRU می‌باشد و می‌تواند برای جایگزینی انتخاب شود. با این وجود، توجه داشته باشید که تضمینی بر یکتایی این اعداد وجود ندارد. می‌توانیم تمامی صفحاتی را که کوچکترین مقدار را دارند برای جایگزینی (مبادله به بیرون) انتخاب کنیم و یا برای انتخاب از میان آنها از روش FIFO استفاده کنیم.

البته، تعداد بیت‌های سابقه می‌تواند متغیر باشد و طوری انتخاب می‌شود (بسته به سخت‌افزار موجود) که به روز رسانی آن با سرعت بیشتری انجام شود. در حد نهایت می‌توان این عدد را به صفر کاهش داد که فقط خود بیت مراجعه باقی می‌ماند. این الگوریتم، **الگوریتم جایگزینی صفحه‌ی شانس دوم** نامیده می‌شود.

۱۰-۴-۵-۲ الگوریتم شانس دوم

الگوریتم پایه‌ای جایگزینی شانس دوم، یک الگوریتم جایگزینی FIFO است. با این وجود، زمانی که صفحه‌ای برای جایگزینی انتخاب می‌شود، بیت مراجعه‌ی آن را بررسی می‌کنیم. اگر مقدار آن ۰ باشد، صفحه برای جایگزینی انتخاب می‌شود؛ اما اگر بیت مراجعه‌ی آن ۱ باشد، به صفحه یک شانس دوم می‌دهیم و به ترتیب FIFO به سراغ صفحه‌ی بعدی می‌رویم. لحظه‌ای که یک صفحه شانس دوم می‌گیرد، بیت مراجعه‌ی آن پاک شده و زمان ورود آن به زمان فعلی مقداردهی می‌گردد. بنابراین، صفحه‌ای که شانس دوم گرفت، جایگزین نخواهد شد تا این که صفحات دیگر برای جایگزینی انتخاب شوند (یا شانس دوم بگیرند). به علاوه، اگر صفحه‌ای به قدر کافی مراجعه داشته باشد طوری که بیت مراجعه‌اش را یک نگه بدارد، هرگز برای جایگزینی انتخاب نخواهد شد.

یک روش پیاده‌سازی الگوریتم شانس دوم (که گاهاً به نام الگوریتم ساعت از آن یاد می‌شود) به صورت صف چرخشی است. یک اشاره‌گر (یعنی، عقربه‌ی ساعت) صفحه‌ای را که در نوبت بعدی برای جایگزینی انتخاب می‌شود، نشان می‌دهد. لحظه‌ای که قابی لازم می‌شود، اشاره‌گر به جلو می‌رود تا صفحه‌ای را با بیت مراجعه‌ی ۰ بیابد. همچنان که اشاره‌گر پیش می‌رود، بیت‌های مراجعه را پاک می‌کند (شکل ۱۰-۱۷). به محض این که یک صفحه‌ی قربانی پیدا شود، صفحه جایگزین می‌شود و صفحه‌ی جدیدالورود در همان محل صفحه‌ی قبلی در صف چرخشی درج می‌شود. توجه داشته باشید که در بدترین حالت (زمانی که بیت مراجعات تمامی صفحات یک باشند) ممکن است اشاره‌گر، کل صف را دور بزند و با این کار به تمامی صفحات شانس دوم بدهد. در این صورت، پیش از این که صفحه‌ی بعدی را برای جایگزینی انتخاب کند بیت مراجعات تمامی صفحات را پاک می‌کند. اگر بیت مراجعات همگی صفحات یک باشند، جایگزینی شانس دوم به جایگزینی FIFO بدل می‌شود.

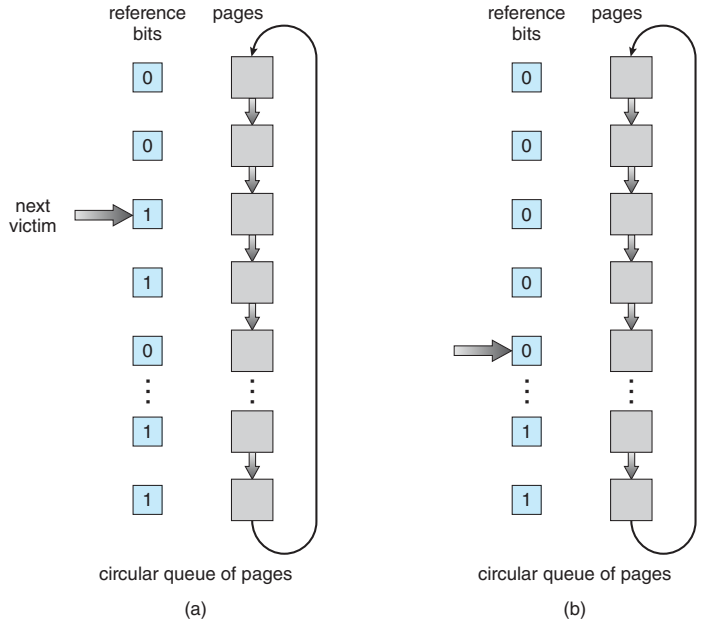
۱۰-۴-۵-۳ الگوریتم شانس دوم بهبودیافته

با در نظر گرفتن بیت مراجعه و بیت تغییر (که در بخش ۱۰-۴-۱ عنوان شد) به صورت یک زوج‌مرتب می‌توانیم الگوریتم شانس دوم را بهبود ببخشیم. با این دو بیت، چهار رده‌ی زیر امکان‌پذیر است:

۱- (0, 0) اخیراً نه استفاده شده و نه تغییر کرده است□بهترین صفحه برای جایگزینی

۲- (0, 1) اخیراً استفاده نداشته، اما تغییر کرده است□جایگزین چندان مناسبی نیست، چون پیش از جایگزینی لازم است که صفحه در دیسک نوشته شود.

۳- (1, 0) اخیراً استفاده شده، اما دست‌نخورده است شاید به زودی باز هم مورد استفاده قرار بگیرد.
۴- (1, 1) اخیراً استفاده داشته و تغییر یافته است شاید به زودی باز هم مورد استفاده قرار بگیرد و پیش از جایگزینی لازم است که صفحه در حافظه‌ی ثانویه نوشته شود.



شکل ۱۰-۱۷ الگوریتم جایگزینی صفحه‌ی شانس دوم (ساعت).

هر صفحه در یکی از این رده‌ها قرار می‌گیرد. زمانی که قرار است جایگزینی صفحه صورت بگیرد، از همان طرح الگوریتم ساعت استفاده می‌کنیم؛ اما به جای این که بررسی کنیم صفحه‌ای که به آن اشاره داریم بیت مراجعه‌اش ۱ است یا خیر، رده‌ای را که صفحه به آن تعلق دارد بررسی می‌کنیم. اولین صفحه‌ای را که در پایین‌ترین رده‌ی ناتهی یافتیم برای جایگزینی انتخاب می‌کنیم. توجه داشته باشید که امکان دارد پیش از این که صفحه‌ای را برای جایگزینی پیدا کنیم صف چرخشی را چندین بار پیمایش کنیم. تفاوت اصلی میان این الگوریتم و الگوریتم ساده‌تر ساعت آن است که در اینجا، برتری را به صفحاتی می‌دهیم که تغییر یافته‌اند تا دفعات مورد نیاز I/O را کاهش دهیم.

۱۰-۴-۶ جایگزینی صفحه‌ی مبتنی بر شمارش

الگوریتم‌های فراوان دیگری هم هستند که می‌توانند برای جایگزینی صفحه استفاده شوند. برای مثال، می‌توانیم شمارنده‌ای برای تعداد مراجعات هر یک از صفحات نگه‌داری کنیم و دو طرح زیر را برای جایگزینی صفحه به کار ببریم.